Integration / MCP-style Interface Specification

1) Decision

We document the **support-request capability** as an MCP-style interface rather than building a network server. It keeps the project within scope, and the tool contracts from Week 4 already map almost one-to-one onto MCP's `tools/list` format. The specification is generated from the live tool registry by `scripts/export_mcp_spec.py` into `docs/interoperability/campusconnect-mcp-tools.json`, so the documentation cannot drift from the code.

Note: the course books do not cover MCP in depth. The mapping below follows the public MCP specification (tools with `inputSchema`, optional `outputSchema` and `annotations`); please verify field names against the current spec at modelcontextprotocol.io before submission.

2) Capability

Item	Value
Interface name	`campusconnect-support` v1.0.0
Purpose	Let an authorised agent look up lecture venues and create, check and submit student support requests
Shape	MCP `tools/list` and `tools/call` over JSON-RPC 2.0 (documented; not exposed on a network)
Identity	Taken from the application session, never from tool arguments
Closed world	No internet, no third-party calls (`openWorldHint: false`)

3) Tools

Tool	Inputs	Output	Auth	Side effect	Approval
`lookup_lecture_venue`	`course_code` (`^[A-Z]{3}\d{4}$`), optional `day` (Mon–Fri)	`count`, `matches[]` (course, day, start, end, venue), `source`	public	read	no
`get_support_request_status`	`case_ref`	request fields (no student ID)	signed-in student, owner only	read	no
`create_support_request_draft`	`category` (registration, fees, timetable, accounts,	request fields + `duplicate_of_existing_draft`	signed-in student	low-risk write (draft,	no

Tool	Inputs	Output	Auth	Side effect	Approval
	other), `summary` (10–500 chars)			invisible to staff)	
`submit_support_request`	`case_ref`	request fields	signed-in student, owner only	high-impact write (notifies staff)	**required**, one-time, argument-bound

Each tool also carries `failure_behaviour` (error code → what happens), exported under the `x-campusconnect` key.

4) Annotations

MCP tool annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`) are exported for each tool. They are **hints for clients, not security controls**; the real enforcement is in our executor (C.5).

5) Security boundary

Trust zones: the model and any external MCP client are **untrusted**; the executor, approval gate and data layer are **trusted**.

Every call, whoever makes it, passes this deterministic pipeline in `Executor.execute`:

1. Is the tool in the allow-list?
2. Do the arguments validate against the JSON Schema (extra fields rejected)?
3. Is the caller authenticated and authorised for this record (owner check)?
4. If high impact, is there a granted, unexpired, single-use approval bound to *these exact arguments* (SHA-256 fingerprint, 300 s TTL)?
5. Run the handler (bounded retries for reads only).
6. Does the output match the output schema? Over-sharing or malformed output is discarded.
7. Write an audit entry.

Errors returned to the caller contain a code and a safe message only, never stack traces.

Threat	Control
Prompt injection via tool results or documents	Prompt rule treats them as data; allow-list and approval gate limit what an injected instruction could do
Confused deputy (acting on another student's case)	Identity from the session; owner check on every case tool
Approval replay or reuse for a different action	Approvals are single-use and bound to student, tool and arguments

Threat	Control
Duplicate staff notifications after a failure	No automatic retry for writes
Data over-sharing	Output schema with `additionalProperties: false`; student ID never returned

6) Example

`tools/call` request an agent might send:

```
{"jsonrpc":"2.0","id":7,"method":"tools/call",
 "params":{"name":"submit_support_request","arguments":{"case_ref":"CC-2026-00001"}}}
```

Without a prior approval the result is `{"ok": false, "error_code": "APPROVAL_REQUIRED", ...}` and nothing is sent.
